

# Chapter 8 — Lab Hands-On

## Student Records Manager with Vue 3, Axios, Express and MySQL

Cross Platform Application Development · Faculty of Computing, Universiti Teknologi Malaysia

| Field          | Details                                                               |
|----------------|-----------------------------------------------------------------------|
| Course         | Web Application Development                                           |
| Chapter        | 8 — Connecting Vue to Backend                                         |
| Lab time       | 2 hours (in-lab) + ~1 hour self-study                                 |
| Lab assessment | 4% (Lab Exercise component)                                           |
| Software       | Laragon (MySQL), Node.js 18+, npm 9+, VS Code, Postman or REST Client |
| Architecture   | Vue 3 (Vite, port 5173) ⇄ Express API (port 3000) ⇄ MySQL (port 3306) |
| Companion      |                                                                       |

### 1. Lab Objectives

By the end of this lab you will be able to:

1. Create a MySQL database, table, and seed data using a simple SQL script in Laragon.
2. Build a small Node.js + Express REST API that talks to MySQL using the mysql2 driver.
3. Scaffold a Vue 3 single-page application using Vite, with a clean component structure.
4. Use Axios with a single shared instance plus request and response interceptors.
5. Implement a Vue form with v-model, modifiers, and custom validation rules.
6. Perform full CRUD against the API and debug HTTP traffic in the browser DevTools.

### 2. Prerequisites & Tools

#### Required

- Laragon (Full edition) with MySQL service running.
- Node.js 18 or newer (verify: node -v).
- npm 9 or newer (verify: npm -v).
- A modern browser (Chrome, Edge or Firefox) with DevTools.
- VS Code (recommended) with the Volar extension for Vue.
- Postman or the VS Code REST Client extension for API testing.

## Prior knowledge

- Chapters 1–7: HTML/CSS, JavaScript ES6+, Vue 3 basics, components, reactivity.
- Basic SQL: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.
- Comfort with the terminal and running npm commands.

### Lab tip

Open Laragon and click Start All before you begin. The Apache and MySQL services must show green. If MySQL fails to start, port 3306 is probably already in use — close the conflicting service or change Laragon's MySQL port.

## 3. Project Structure

By the end of this lab your project folder will look like this:

```
student-records-app/
├── server/                ← Express + MySQL backend
│   ├── package.json
│   ├── index.js          ← API routes
│   └── db.js             ← MySQL connection pool
├── sql/
│   └── schema.sql        ← CREATE DATABASE + seed data
├── index.html            ← Vite entry HTML
├── package.json          ← Vue dependencies and scripts
├── vite.config.js        ← Vite configuration
├── src/
│   ├── main.js           ← Vue entry point
│   ├── App.vue           ← root component (orchestrates the app)
│   ├── api/
│   │   └── studentApi.js ← Axios instance + CRUD functions
│   ├── assets/
│   │   └── main.css      ← global styles
│   └── components/
│       ├── StudentForm.vue
│       └── StudentList.vue
```

### Two terminals, two services

Throughout this lab you always need TWO terminals: one running the Express API (npm run server, port 3000) and one running Vite (npm run dev, port 5173). MySQL itself runs under Laragon — you do not start it from the terminal.

## 4. Part 1 — Scaffolding the Vue 3 Project

### Step 1.1 — Create the project folder

Open a terminal and run:

```
mkdir student-records-app
```

```
cd student-records-app
npm init -y
```

### Step 1.2 — Install Vue 3, Vite, and Axios

```
npm install vue axios
npm install -D vite @vitejs/plugin-vue
```

Vue and Axios are runtime dependencies; Vite (and its Vue plugin) are dev dependencies.

### Step 1.3 — Create vite.config.js

In the project root, create a file named vite.config.js with the following content:

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: { port: 5173, open: true }
})
```

### Step 1.4 — Create index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Student Records Manager | Chapter 8</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
```

### Step 1.5 — Create src/main.js and a placeholder App.vue

Create the src/ folder. Inside it, create main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import './assets/main.css'

createApp(App).mount('#app')
```

Then create src/App.vue with a placeholder:

```
<script setup></script>

<template>
  <h1>Student Records Manager</h1>
  <p>Scaffolding complete.</p>
</template>
```

Create an empty src/assets/main.css for now — we will style the app at the end.

## Step 1.6 — Add npm scripts

Open the project-root package.json and replace the scripts section with:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview",  
  "server": "node server/index.js"  
}
```

## Step 1.7 — Test the dev server

```
npm run dev
```

Open <http://localhost:5173> in the browser. You should see the placeholder heading. Stop the server with Ctrl+C.

### Checkpoint 1

If the page loads, Part 1 is complete. Commit your code now if you are using git.

## 5. Part 2 — Setting Up the MySQL Database

### Step 2.1 — Open the Laragon MySQL terminal

In Laragon, right-click the tray icon and choose Tools → MySQL → New MySQL session, or simply click the "Database" button in the main window. This opens HeidiSQL (a graphical MySQL client) already connected to your local server as the root user with an empty password.

#### Heads up — root password

Laragon ships with no password for the root user out of the box. For this lab that is fine because the database is only on your laptop. NEVER deploy an empty-password MySQL to a real server.

### Step 2.2 — Create the schema file

In your project root, create a folder named sql/ and a file inside it named schema.sql:

```
-- sql/schema.sql  
CREATE DATABASE IF NOT EXISTS student_records  
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
USE student_records;  
  
DROP TABLE IF EXISTS students;  
  
CREATE TABLE students (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  matricNo    VARCHAR(10) NOT NULL UNIQUE,
```

```
name      VARCHAR(120) NOT NULL,
course    VARCHAR(150) NOT NULL,
faculty   VARCHAR(30)  NOT NULL,
gpa       DECIMAL(3,2) NOT NULL,
email     VARCHAR(120) NOT NULL,
year      TINYINT     NOT NULL,
active    TINYINT(1)  NOT NULL DEFAULT 1,
CHECK (gpa >= 0 AND gpa <= 4),
CHECK (year BETWEEN 1 AND 6)
) ENGINE=InnoDB;

INSERT INTO students
(matricNo, name, course, faculty, gpa, email, year, active)
VALUES
('A21CS0001', 'Ahmad Zulkarnain bin Hassan',
 'Bachelor of Computer Science (Software Engineering)',
 'FSKSM', 3.75, 'ahmad.zulkarnain@graduate.utm.my', 3, 1),
('A21CS0002', 'Nur Aisyah binti Abdullah',
 'Bachelor of Computer Science (Data Engineering)',
 'FSKSM', 3.92, 'nur.aisyah@graduate.utm.my', 3, 1),
('A21CS0003', 'Muhammad Faris bin Razak',
 'Bachelor of Computer Science (Cybersecurity)',
 'FSKSM', 3.45, 'm.faris@graduate.utm.my', 2, 1),
('A21IT0004', 'Siti Khadijah binti Ibrahim',
 'Bachelor of Information Technology',
 'FSKSM', 3.68, 'siti.khadijah@graduate.utm.my', 4, 1),
('A21CS0005', 'Tan Wei Ming',
 'Bachelor of Computer Science (Software Engineering)',
 'FSKSM', 3.81, 'tan.weiming@graduate.utm.my', 2, 1);
```

### Step 2.3 — Run the schema

In HeidiSQL: File → Load SQL file → choose your schema.sql, then press F9 (or click the blue triangle) to execute. You should see one INSERT statement per row in the messages panel.

Refresh the database list (F5). You should now see student\_records → students with 5 rows.

### Step 2.4 — Smoke-test from the command line (optional)

Open the Laragon Terminal (right-click tray → Terminal) and run:

```
mysql -u root student_records -e "SELECT id, matricNo, name, gpa FROM students;"
```

You should see five rows in the terminal. If the command is "not recognised", make sure you opened the Laragon Terminal (which pre-loads MySQL onto the PATH), not the regular Windows terminal.

#### Checkpoint 2

Five rows visible in HeidiSQL and/or via the mysql CLI. Your database is ready.

## 6. Part 3 — Building the Express API

### Step 3.1 — Create the backend folder and install dependencies

From the project root:

```
mkdir server
cd server
npm init -y
npm install express mysql2 cors
npm install -D nodemon
cd ..
```

What each package does:

- **express** — minimal web framework for routing and JSON parsing.
- **mysql2** — fast MySQL driver with a Promise-based API and built-in support for prepared statements.
- **cors** — middleware that lets the Vite dev server (port 5173) call the API (port 3000) without CORS errors.
- **nodemon** — dev-only; restarts the API automatically when you save a change.

### Step 3.2 — Create server/db.js (connection pool)

```
// server/db.js
const mysql = require('mysql2/promise')

const pool = mysql.createPool({
  host: 'localhost',
  port: 3306,
  user: 'root',
  password: '', // Laragon default; do NOT do this in production
  database: 'student_records',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
})

module.exports = pool
```

#### Why a pool?

`createPool` reuses a small set of connections across many requests. `createConnection` opens one connection per call and is much slower under load. Always use a pool for a REST API.

### Step 3.3 — Create server/index.js (routes)

```
// server/index.js
const express = require('express')
const cors = require('cors')
const pool = require('./db')

const app = express()
app.use(cors()) // allow requests from http://localhost:5173
```

```
app.use(express.json())      // parse application/json request bodies

// --- Health check ---
app.get('/', (req, res) => res.json({ status: 'ok' }))

// --- READ list (with optional search & sort) ---
app.get('/students', async (req, res) => {
  try {
    const { q, sortBy, order } = req.query
    let sql = 'SELECT * FROM students'
    const params = []

    if (q) {
      sql += ` WHERE name LIKE ?
              OR matricNo LIKE ?
              OR email LIKE ?
              OR course LIKE ?`
      const like = `%${q}%`
      params.push(like, like, like, like)
    }

    const allowedSort = ['name', 'matricNo', 'gpa', 'year']
    if (sortBy && allowedSort.includes(sortBy)) {
      const direction = order === 'desc' ? 'DESC' : 'ASC'
      sql += ` ORDER BY ${sortBy} ${direction}`
    }

    const [rows] = await pool.query(sql, params)
    res.json(rows)
  } catch (err) {
    console.error(err)
    res.status(500).json({ error: 'Database error' })
  }
})

// --- READ single ---
app.get('/students/:id', async (req, res) => {
  try {
    const [rows] = await pool.query(
      'SELECT * FROM students WHERE id = ?', [req.params.id])
    if (!rows.length) return res.status(404).json({ error: 'Not found' })
    res.json(rows[0])
  } catch (err) {
    res.status(500).json({ error: 'Database error' })
  }
})

// --- CREATE ---
app.post('/students', async (req, res) => {
  try {
    const { matricNo, name, course, faculty, gpa, email, year, active } = req.body
    const [r] = await pool.query(
      `INSERT INTO students
       (matricNo, name, course, faculty, gpa, email, year, active)`
```

```

    VALUES (?, ?, ?, ?, ?, ?, ?, ?)` ,
    [matricNo, name, course, faculty, gpa, email, year, active ? 1 : 0]
  )
  res.status(201).json({ id: r.insertId, ...req.body })
} catch (err) {
  if (err.code === 'ER_DUP_ENTRY') {
    return res.status(409).json({ error: 'Matric number already exists' })
  }
  console.error(err)
  res.status(500).json({ error: 'Database error' })
}
})

// --- UPDATE ---
app.put('/students/:id', async (req, res) => {
  try {
    const { matricNo, name, course, faculty, gpa, email, year, active } = req.body
    const [r] = await pool.query(
      `UPDATE students SET
        matricNo=?, name=?, course=?, faculty=?,
        gpa=?, email=?, year=?, active=?
      WHERE id=?`,
      [matricNo, name, course, faculty, gpa, email, year,
        active ? 1 : 0, req.params.id]
    )
    if (!r.affectedRows) return res.status(404).json({ error: 'Not found' })
    res.json({ id: Number(req.params.id), ...req.body })
  } catch (err) {
    res.status(500).json({ error: 'Database error' })
  }
})

// --- DELETE ---
app.delete('/students/:id', async (req, res) => {
  try {
    const [r] = await pool.query(
      'DELETE FROM students WHERE id = ?', [req.params.id])
    if (!r.affectedRows) return res.status(404).json({ error: 'Not found' })
    res.json({ deleted: true })
  } catch (err) {
    res.status(500).json({ error: 'Database error' })
  }
})

const PORT = 3000
app.listen(PORT, () =>
  console.log(`API running at http://localhost:${PORT}`))

```

### Why prepared statements?

Every value we put into a query goes through a "?" placeholder, not via string concatenation. This prevents SQL injection — even if a student types `""; DROP TABLE students; --` in the search box, the driver escapes it. Never build SQL with string templates.



### Step 3.4 — Start the API and smoke-test

From the project root, in a new terminal:

```
npm run server
```

You should see:

```
API running at http://localhost:3000
```

Open these URLs in the browser or test with Postman / REST Client:

- **`http://localhost:3000/students`** — should return a JSON array of 5 students.
- **`http://localhost:3000/students/1`** — returns the first student record.
- **`http://localhost:3000/students?q=aisyah`** — server-side search.
- **`http://localhost:3000/students?sortBy=gpa&order=desc`** — sort by GPA, highest first.

### Step 3.5 — Test POST / PUT / DELETE with REST Client

Create a file named `test.http` in the project root (the VS Code REST Client extension recognises this):

```
### Create
POST http://localhost:3000/students
Content-Type: application/json

{
  "matricNo": "A21CS9999",
  "name": "Test Student",
  "course": "BSc CS",
  "faculty": "FSKSM",
  "gpa": 3.50,
  "email": "test@graduate.utm.my",
  "year": 1,
  "active": true
}

### Update (replace <id> with the id returned above)
PUT http://localhost:3000/students/6
Content-Type: application/json

{
  "matricNo": "A21CS9999",
  "name": "Test Student (updated)",
  "course": "BSc CS",
  "faculty": "FSKSM",
  "gpa": 3.80,
  "email": "test@graduate.utm.my",
  "year": 2,
  "active": true
}

### Delete
DELETE http://localhost:3000/students/6
```

**Checkpoint 3**

All four verbs return JSON with the expected status codes (200 / 201 / 404). Verify in HeidiSQL that the row count actually changes after each call.

## 7. Part 4 — Creating the Axios API Service

### Step 4.1 — Create src/api/studentApi.js

Create the folder src/api/ and inside it a new file named studentApi.js:

```
import axios from 'axios'

// 1. Pre-configured Axios instance
const apiClient = axios.create({
  baseURL: 'http://localhost:3000',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json'
  }
})

// 2. Request interceptor – runs before every outgoing request
apiClient.interceptors.request.use(
  (config) => {
    console.log(`[API] ${config.method?.toUpperCase()} ${config.url}`)
    return config
  },
  (error) => Promise.reject(error)
)

// 3. Response interceptor – runs on every response
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response) {
      console.error(`[API ERROR] ${error.response.status} - ${error.response.statusText}`)
    } else if (error.request) {
      console.error(`[API ERROR] No response from server (is the API running?)`)
    } else {
      console.error(`[API ERROR]', error.message)
    }
    return Promise.reject(error)
  }
)

// 4. CRUD helpers
export const getStudents = (params = {}) =>
  apiClient.get('/students', { params })
export const getStudent = (id) => apiClient.get(`/students/${id}`)
```

```
export const createStudent = (s) => apiClient.post('/students', s)
export const updateStudent = (id, s) => apiClient.put(`/students/${id}`, s)
export const deleteStudent = (id) => apiClient.delete(`/students/${id}`)

export default apiClient
```

### Why a service module?

Components should never import axios directly. Centralising API calls means: (1) one place to change the baseURL when you deploy; (2) easy to mock in tests; (3) the UI layer stays clean and declarative.

## 8. Part 5 — Building the Form with Validation

### Step 5.1 — Component skeleton

Create src/components/StudentForm.vue:

```
<script setup>
import { ref, watch, computed } from 'vue'

const props = defineProps({
  editingStudent: { type: Object, default: null }
})
const emit = defineEmits(['save', 'cancel'])

const emptyForm = () => ({
  matricNo: '', name: '', course: '', faculty: '',
  gpa: '', email: '', year: 1, active: true
})

const form = ref(emptyForm())
const errors = ref({})

watch(() => props.editingStudent, (val) => {
  form.value = val ? { ...val } : emptyForm()
  errors.value = {}
}, { immediate: true })

const isEditing = computed(() => Boolean(props.editingStudent))
</script>
```

### Step 5.2 — Add validation logic

Append to the same <script setup> block:

```
const matricRegex = /^[A-Z][0-9]{2}[A-Z]{2}[0-9]{4}$/
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/

function validate() {
  const e = {}
```

```

    if (!form.value.matricNo.trim())
      e.matricNo = 'Matric number is required.'
    else if (!matricRegex.test(form.value.matricNo.trim().toUpperCase()))
      e.matricNo = 'Format: A21CS0001 (1 letter, 2 digits, 2 letters, 4 digits).'

    if (!form.value.name.trim() || form.value.name.trim().length < 3)
      e.name = 'Name must be at least 3 characters.'

    if (!emailRegex.test(form.value.email.trim()))
      e.email = 'Please enter a valid email address.'

    const gpa = Number(form.value.gpa)
    if (Number.isNaN(gpa) || gpa < 0 || gpa > 4)
      e.gpa = 'GPA must be between 0.00 and 4.00.'

    if (!form.value.course.trim()) e.course = 'Course is required.'
    if (!form.value.faculty.trim()) e.faculty = 'Faculty is required.'

    errors.value = e
    return Object.keys(e).length === 0
  }

function onSubmit() {
  if (!validate()) return
  emit('save', {
    ...form.value,
    matricNo: form.value.matricNo.trim().toUpperCase(),
    name:     form.value.name.trim(),
    email:    form.value.email.trim(),
    gpa:      Number(form.value.gpa),
    year:     Number(form.value.year)
  })
  if (!isEditing.value) form.value = emptyForm()
}

function onCancel() {
  emit('cancel')
  form.value = emptyForm()
  errors.value = {}
}

```

### Step 5.3 — Template with v-model modifiers

Add the <template> block. Notice .trim and .number modifiers, and the inline error spans:

```

<template>
  <form @submit.prevent="onSubmit" class="student-form">
    <h3>{{ isEditing ? 'Edit Student' : 'Add New Student' }}</h3>

    <label>Matric No
      <input v-model.trim="form.matricNo" placeholder="A21CS0001" />
      <span v-if="errors.matricNo" class="err">{{ errors.matricNo }}</span>
    </label>

```

```

<label>Full Name
  <input v-model.trim="form.name" />
  <span v-if="errors.name" class="err">{{ errors.name }}</span>
</label>

<label>Course
  <input v-model.trim="form.course" />
  <span v-if="errors.course" class="err">{{ errors.course }}</span>
</label>

<label>Faculty
  <select v-model="form.faculty">
    <option value="">-- Select --</option>
    <option>FSKSM</option>
    <option>FKE</option>
    <option>FAB</option>
  </select>
  <span v-if="errors.faculty" class="err">{{ errors.faculty }}</span>
</label>

<label>Email
  <input v-model.trim="form.email" type="email" />
  <span v-if="errors.email" class="err">{{ errors.email }}</span>
</label>

<label>GPA
  <input v-model.number="form.gpa" type="number" step="0.01"
    min="0" max="4" />
  <span v-if="errors.gpa" class="err">{{ errors.gpa }}</span>
</label>

<label>Year
  <select v-model.number="form.year">
    <option v-for="y in [1,2,3,4,5,6]" :key="y" :value="y">{{ y }}</option>
  </select>
</label>

<label class="check">
  <input type="checkbox" v-model="form.active" /> Active student
</label>

<div class="actions">
  <button type="submit">{{ isEditing ? 'Update' : 'Add' }}</button>
  <button v-if="isEditing" type="button" @click="onCancel">Cancel</button>
</div>
</form>
</template>

```

## 9. Part 6 — Building the List Component

### Step 6.1 — Create src/components/StudentList.vue

```
<script setup>
```

```

defineProps({ students: { type: Array, required: true } })
const emit = defineEmits(['edit', 'delete'])
</script>

<template>
  <div v-if="!students.length" class="empty">
    No students found. Add one above or clear the search filter.
  </div>

  <table v-else class="student-table">
    <thead>
      <tr>
        <th>Matric</th><th>Name</th><th>Course</th>
        <th>GPA</th><th>Year</th><th>Status</th><th>Actions</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="s in students" :key="s.id">
        <td><code>{{ s.matricNo }}</code></td>
        <td>
          <strong>{{ s.name }}</strong>
          <div class="muted">{{ s.email }}</div>
        </td>
        <td>{{ s.course }}</td>
        <td>{{ Number(s.gpa).toFixed(2) }}</td>
        <td>{{ s.year }}</td>
        <td>
          <span :class="s.active ? 'badge ok' : 'badge no'">
            {{ s.active ? 'Active' : 'Inactive' }}
          </span>
        </td>
        <td>
          <button @click="emit('edit', s)">Edit</button>
          <button class="danger" @click="emit('delete', s.id)">Delete</button>
        </td>
      </tr>
    </tbody>
  </table>
</template>

```

#### A note on the GPA field

MySQL returns DECIMAL columns as strings (e.g. "3.75") to avoid floating-point loss. That is why the template uses `Number(s.gpa).toFixed(2)` before display — otherwise you would see "3.7500".

## 10. Part 7 — Wiring Everything Together in App.vue

### Step 7.1 — Replace App.vue

```

<script setup>
import { ref, onMounted } from 'vue'
import StudentForm from './components/StudentForm.vue'
import StudentList from './components/StudentList.vue'
import {

```

```
    getStudents, createStudent, updateStudent, deleteStudent
  } from './api/studentApi.js'

  const students      = ref([])
  const editingStudent = ref(null)
  const loading        = ref(false)
  const errorMsg       = ref('')

  async function load() {
    loading.value = true
    errorMsg.value = ''
    try {
      const { data } = await getStudents()
      students.value = data
    } catch (e) {
      errorMsg.value = 'Failed to load students. Is the API running on :3000?'
    } finally {
      loading.value = false
    }
  }

  async function handleSave(payload) {
    try {
      if (editingStudent.value) {
        await updateStudent(editingStudent.value.id, payload)
        editingStudent.value = null
      } else {
        await createStudent(payload)
      }
      await load()
    } catch (e) {
      errorMsg.value = 'Save failed. Check the console for details.'
    }
  }

  function handleEdit(s) { editingStudent.value = { ...s } }
  function handleCancel() { editingStudent.value = null }

  async function handleDelete(id) {
    if (!confirm('Delete this student? This cannot be undone.')) return
    try {
      await deleteStudent(id)
      await load()
    } catch {
      errorMsg.value = 'Delete failed.'
    }
  }

  onMounted(load)
</script>

<template>
  <header>
    <h1>Student Records Manager</h1>
  </header>
</template>
```

```

    <p class="subtitle">Vue 3 · Axios · Express · MySQL</p>
  </header>

  <main>
    <p v-if="loading" class="loading">Loading...</p>
    <p v-if="errorMsg" class="error">{{ errorMsg }}</p>

    <StudentForm
      :editingStudent="editingStudent"
      @save="handleSave"
      @cancel="handleCancel" />

    <StudentList
      :students="students"
      @edit="handleEdit"
      @delete="handleDelete" />
  </main>
</template>

```

## Step 7.2 — Style it

Paste this into `src/assets/main.css`:

```

body { font-family: system-ui, sans-serif; background: #f8fafc;
  margin: 0; color: #0f172a; }
header { background: #0f766e; color: white; padding: 1.2rem 2rem; }
header h1 { margin: 0; font-size: 1.6rem; }
header .subtitle { margin: 0.2rem 0 0 0; opacity: 0.85; font-size: 0.9rem; }
main { max-width: 1100px; margin: 1.5rem auto; padding: 0 1rem;
  display: grid; gap: 1.5rem; }

.student-form, .student-table {
  background: white; padding: 1rem 1.2rem;
  border-radius: 8px; box-shadow: 0 1px 3px rgba(0,0,0,0.08);
}
.student-form label { display: block; margin: 0.5rem 0; font-size: 0.9rem; }
.student-form input, .student-form select {
  display: block; width: 100%; padding: 0.45rem 0.6rem; margin-top: 0.25rem;
  border: 1px solid #cbd5e1; border-radius: 4px;
}
.student-form .err { color: #b91c1c; font-size: 0.78rem; }
.student-form button { padding: 0.45rem 1rem; border: 0;
  background: #0f766e; color: white; border-radius: 4px; cursor: pointer; }
.student-form button.danger,
.student-table .danger { background: #b91c1c; }

table { width: 100%; border-collapse: collapse; font-size: 0.9rem; }
th, td { text-align: left; padding: 0.55rem 0.6rem;
  border-bottom: 1px solid #e2e8f0; vertical-align: top; }
th { background: #f1f5f9; }
code { background: #ccfbf1; color: #115e59;
  padding: 0.15rem 0.4rem; border-radius: 4px; }
.muted { font-size: 0.78rem; color: #64748b; }
.badge { display: inline-block; padding: 0.15rem 0.5rem;
  border-radius: 999px; font-size: 0.75rem; }

```



```
.badge.ok { background: #d1fae5; color: #065f46; }  
.badge.no { background: #fee2e2; color: #991b1b; }  
.error { background: #fee2e2; color: #991b1b; padding: 0.6rem 1rem;  
        border-radius: 6px; }  
.loading { color: #64748b; font-style: italic; }
```

## 11. Part 8 — Run and Verify End-to-End

You now need three things running:

7. Laragon → MySQL service (green light).
8. Express API on port 3000: `npm run server`
9. Vite dev server on port 5173: `npm run dev`

Open `http://localhost:5173` and perform this acceptance test:

10. You see the 5 seeded students from MySQL.
11. Add a new student with matric A21CS0010. It appears in the table.
12. Click Edit on the row, change the GPA to 3.99, click Update. The row updates.
13. Click Delete. The row disappears.
14. In HeidiSQL, refresh the students table — the data matches the UI exactly.
15. Open the browser DevTools → Network tab. Each action you take above produces one HTTP request to `localhost:3000` with the expected method.

### Checkpoint 4 — Final

If all six steps work and HeidiSQL agrees with the UI, the lab is complete. The data is now truly persisted in MySQL — restart everything and the changes survive.

## 12. Common Errors and How to Fix Them

| Error you see                                       | Probable cause and fix                                                                                                                                                       |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ECONNREFUSED 127.0.0.1:3306                         | MySQL service is not running. Open Laragon and click Start All until the MySQL lamp turns green.                                                                             |
| ER_BAD_DB_ERROR: Unknown database "student_records" | The schema.sql script did not run. Re-execute schema.sql in HeidiSQL.                                                                                                        |
| ER_ACCESS_DENIED_ERROR for user "root"              | You set a password in MySQL. Either reset it to empty in Laragon (Menu → MySQL → Set root password → leave blank) or update password: in <code>server/db.js</code> to match. |
| CORS error in the browser console                   | You forgot <code>app.use(cors())</code> in <code>server/index.js</code> , or the API server is not running. Restart it.                                                      |

| Error you see                                                   | Probable cause and fix                                                                                |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| POST returns 500 with "ER_DATA_TOO_LONG"                        | A column value exceeds its VARCHAR limit. Check the matricNo (10 chars) or course (150 chars) length. |
| axios returns Network Error and there is nothing in the API log | The API server crashed. Check the terminal where npm run server runs — there will be a stack trace.   |
| GPA shows as "3.7500" in the UI                                 | You forgot Number(s.gpa).toFixed(2) in the template. MySQL returns DECIMAL as a string.               |

### 13. Extension Challenges (Optional)

Finished the core lab with time to spare? Pick one or more of the challenges below. Each is designed to take 30–60 minutes and to reinforce — not replace — the Chapter 8 learning outcomes. Extensions are NOT required for a full lab mark, but a clean, working extension can recover up to 5 marks on the assignment that follows. Mention any extension you attempted in your README.

| # | Challenge                                       | Difficulty  | Reinforces |
|---|-------------------------------------------------|-------------|------------|
| A | Pagination — return only N rows per page        | Easy        | LO 2, LO 4 |
| B | PATCH endpoint + one-click Active toggle        | Easy        | LO 2, LO 4 |
| C | Server-side validation with 400 responses       | Medium      | LO 3, LO 4 |
| D | Faculty statistics endpoint + Chart.js doughnut | Medium-Hard | LO 1, LO 4 |
| E | Move DB credentials into a .env file (dotenv)   | Easy        | LO 1       |

#### Extension A — Pagination

Add server-side pagination to the GET /students endpoint. Accept two new query parameters: page (1-based, default 1) and size (default 10). Translate them to SQL using LIMIT and OFFSET. Return the page of rows AND a total count so the UI can show "Page 1 of 7".

```
-- Hint (SQL)
SELECT * FROM students
ORDER BY id
LIMIT ? OFFSET ?    -- size, (page - 1) * size

-- Total row count for the UI
SELECT COUNT(*) AS total FROM students;
```

In Vue, add Prev / Next buttons that increment a page ref and re-fetch through Axios. Bonus: wrap the response in { data: rows, total, page, size } and update the page label live as the user navigates.

#### Extension B — PATCH endpoint and active toggle

Add a PATCH `/students/:id` endpoint that only updates the fields present in the request body — never the entire row. Use it from the UI to add a one-click "toggle active" button on each table row that sends a tiny PATCH payload (`{ active: !current }`) instead of a full PUT.

```
// Hint — express side
const allowed = ['matricNo', 'name', 'course', 'faculty', 'gpa', 'email', 'year', 'active']
const entries = Object.entries(req.body).filter(([k]) => allowed.includes(k))
const setClause = entries.map(([k]) => `${k}=?`).join(', ')
const params = entries.map([_, v] => v).concat(req.params.id)
await pool.query(`UPDATE students SET ${setClause} WHERE id=?`, params)
```

Demonstrates partial updates and a smaller HTTP payload. Bonus: update the row in the UI optimistically — flip the badge first, roll back if the request fails.

### Extension C — Server-side validation

Client-side validation is good UX but cannot be trusted. Anyone can call your API with curl. Add a `validate()` helper in `server/index.js` that re-checks the same rules (matric regex, email format, GPA range, year range). On failure, return HTTP 400 with a JSON body shaped like `{ errors: { matricNo: "Format: A21CS0001", gpa: "Must be 0–4" } }`.

In Vue, update the response interceptor: if the error response status is 400, copy `err.response.data.errors` into the form's errors ref so the inline messages turn red. Bonus: also reject duplicate emails (the schema only enforces unique matricNo).

### Extension D — Faculty statistics + chart

Add a new endpoint GET `/students/stats` that returns aggregated counts per faculty:

```
SELECT faculty,
       COUNT(*) AS total,
       AVG(gpa) AS avgGpa,
       MIN(year) AS minYear,
       MAX(year) AS maxYear
FROM   students
GROUP BY faculty
ORDER BY total DESC;
```

In Vue, install Chart.js (npm install chart.js vue-chartjs) and render the result as a doughnut chart of student counts per faculty. Refresh the chart whenever the user creates or deletes a student. Excellent practice for SQL aggregation and reactive data flow.

### Extension E — Environment variables with dotenv

Hard-coding DB credentials in `server/db.js` is fine on your laptop but unacceptable in any real project. Install dotenv (npm install dotenv) and move the credentials into a `.env` file:

```
# .env (NEVER commit this file)
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
```

```
DB_NAME=student_records
// server/db.js
require('dotenv').config()
const pool = mysql.createPool({
  host:      process.env.DB_HOST,
  port:      Number(process.env.DB_PORT),
  user:      process.env.DB_USER,
  password:  process.env.DB_PASSWORD,
  database:  process.env.DB_NAME,
  ...
})
```

Add `.env` to `.gitignore` and create a `.env.example` with placeholder values that you DO commit. This is the "twelve-factor" pattern used by virtually every production Node application.

#### How extensions are marked

For each completed extension, attach a screenshot or short note in your README showing it working (e.g. the Network tab showing `?page=2&size=5`, or the 400 response with field-level errors). Up to 5 bonus marks total across all extensions — pick depth over breadth.

## 14. Mapping to Chapter 8 Learning Outcomes

| LO   | Topic                       | Where you practised it in this lab                                                                                                     |
|------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| LO 1 | Connecting Vue to a Backend | Parts 2, 3 and the <code>apiClient</code> <code>baseUrl</code> in Part 4.                                                              |
| LO 2 | Axios HTTP Client           | Part 4 — single instance, request and response interceptors, <code>async/await</code> CRUD helpers.                                    |
| LO 3 | Form Handling in Vue        | Part 5 — <code>v-model</code> with <code>.trim</code> and <code>.number</code> , <code>validate()</code> function, inline error spans. |
| LO 4 | Persistent CRUD             | Parts 3, 6 and 7 — full create / read / update / delete persisted in MySQL.                                                            |

— End of Lab Handout —